



Lab Manual – 8

Code/Course: 23CSE312/Distributed Systems

Topic: Vector Clock using Singhal–Kshemkalyani Differential Technique

[CO05] [BTL-3]

Student Name: Garlapati Tejaswini

Roll No.: AM.SC.U4CSE23219

Batch/Section: CSE-C

Date: 01/05/2026

In distributed systems, logical clocks are used to capture the causal relationship between events occurring at different processes. A traditional vector clock maintains an array of size n at each process, where n is the total number of processes. However, transmitting the complete vector clock with every message creates communication overhead.

To reduce this overhead, Singhal–Kshemkalyani proposed a differential technique, where only the modified entries of the vector clock are transmitted instead of the full vector. This optimization significantly reduces message size while preserving causal ordering.

1 Aim

To implement Vector Clock synchronization using the Singhal–Kshemkalyani Differential Technique and simulate causal event ordering in a distributed system.

2 Objectives

- Understand the concept of logical clocks and vector clocks.
- Learn the communication overhead problem in standard vector clocks.
- Implement the differential update mechanism proposed by Singhal–Kshemkalyani.
- Simulate internal, send, and receive events among distributed processes.
- Verify causality between events using optimized vector timestamps.

3 Problem Statement

Write a Python program to simulate a distributed system consisting of **three processes** P_1 , P_2 , and P_3 using threads or separate objects.

Each process maintains:

- Local vector clock $VC[i]$
- Last Sent array $LS[i]$
- Last Updated array $LU[i]$

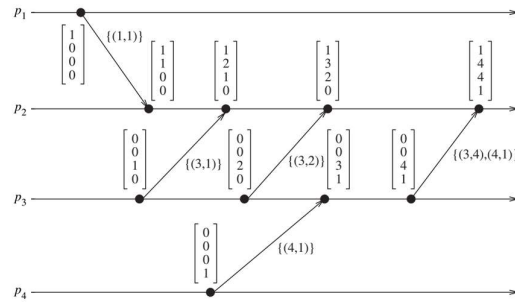


Figure 1: Caption

The program must support the following events and communicate via TCP sockets:

1. Internal Event
2. Send Message
3. Receive Message

During message transmission:

- Compare current vector clock with previously sent values.
- Send only changed entries (differential update).
- Receiver reconstructs the updated vector clock.

4 Input Sequence

Simulate the events as in the attached space-time diagram following event sequence:

5 Expected Output

Display after every event:

- Process performing the event
- Type of event
- Differential entries transmitted
- Updated vector clock of all processes

Example:

P1 Internal Event -> [1,0,0]

P1 sends to P2

Transmitted entries: {(1,1)}

P2 updated clock: [1,0,0]

P2 Internal Event -> [1,1,0]

```
C:\Users\garla\Desktop\Sem 6 Labsheets\Distributed Systems\labsheet8>python -u
"c:\Users\garla\Desktop\Sem 6 Labsheets\Distributed Systems\labsheet8\main.py"
P1 Internal Event -> [1, 0, 0]

P1 sends to P2
Transmitted entries: {0: 2}
P2 updated clock: [2, 1, 0]
P2 Internal Event -> [2, 2, 0]

P2 sends to P3
Transmitted entries: {0: 2, 1: 3}
P3 updated clock: [2, 3, 1]
P3 Internal Event -> [2, 3, 2]

P3 sends to P1
Transmitted entries: {0: 2, 1: 3, 2: 3}
P1 updated clock: [3, 3, 3]
P1 Internal Event -> [4, 3, 3]
```

Singhal–Kshemkalyani technique:

The Singhal–Kshemkalyani technique is an optimization of the traditional vector clock mechanism used in distributed systems to maintain causal ordering of events. In standard vector clocks, every process maintains and transmits a full vector of size equal to the number of processes in the system, which can become inefficient in large-scale systems.

This technique reduces overhead by transmitting only the relevant entries of the vector clock instead of the entire vector. Each process keeps track of only those timestamps that are necessary to preserve causal relationships, thereby minimizing message size and storage requirements while still ensuring correct causal ordering of events across the distributed system.

6 Submission Requirements

- Source code file
- Screenshot of execution output
- Short explanation of Singhal–Kshemkalyani technique

7 Conclusion

The Singhal–Kshemkalyani technique improves vector clock-based synchronization in distributed systems by reducing the amount of timestamp information exchanged between processes. It ensures causal ordering while avoiding the need to send full vector clocks in every message.

Overall, it provides an efficient way to maintain event causality with less communication and storage overhead, making it suitable for large-scale distributed systems.